

stat431_final report R implementation

Jack Liu (jackliu3), Xing Sun (xingsun3), Ning Xu (ningx4), Martin Wang(hanran2)

2026-05-11

```
library(tidyverse)

## -- Attaching core tidyverse packages -----
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
## -- Conflicts -----
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

# =====
# Data cleaning for Bayesian hierarchical used-car price model
# =====
# Goal:
# Prepare used_cars.csv for a Bayesian hierarchical prediction model:
# response: log(price)
# fixed predictors: age, log(mileage), fuel type, transmission,
#                   accident history, clean title
# hierarchical grouping variable: brand

library(tidyverse)

# -----
# 1. Read raw data
# -----

INPUT_PATH <- "used_cars.csv"

car_raw <- read_csv(
  INPUT_PATH,
  na = c("", "NA", "N/A", "NaN", "nan", "null", "NULL"),
  show_col_types = FALSE
)

cat("Raw data dimensions:\n")

## Raw data dimensions:
print(dim(car_raw))

## [1] 4009  12
```

```

cat("\nColumn names:\n")

##
## Column names:
print(names(car_raw))

## [1] "brand"      "model"      "model_year" "milage"     "fuel_type"
## [6] "engine"     "transmission" "ext_col"    "int_col"    "accident"
## [11] "clean_title" "price"

cat("\nFirst few rows:\n")

##
## First few rows:
print(head(car_raw))

## # A tibble: 6 x 12
##   brand    model model_year milage fuel_type engine transmission ext_col int_col
##   <chr>   <chr>      <dbl> <chr>  <chr>    <chr>  <chr>      <chr>  <chr>
## 1 Ford    Util~      2013 51,00~ E85 Flex~ 300.0~ 6-Speed A/T Black  Black
## 2 Hyundai Pali~      2021 34,74~ Gasoline 3.8L ~ 8-Speed Aut~ Moonli~ Gray
## 3 Lexus   RX 3~      2022 22,37~ Gasoline 3.5 L~ Automatic Blue  Black
## 4 INFINITI Q50 ~      2015 88,90~ Hybrid 354.0~ 7-Speed A/T Black  Black
## 5 Audi    Q3 4~      2021 9,835~ Gasoline 2.0L ~ 8-Speed Aut~ Glacie~ Black
## 6 Acura   ILX ~      2016 136,3~ Gasoline 2.4 L~ F Silver Ebony.
## # i 3 more variables: accident <chr>, clean_title <chr>, price <chr>

missing_before <- car_raw %>%
  summarise(across(everything(), ~sum(is.na(.)))) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "missing_count")

cat("\nMissing values before cleaning:\n")

##
## Missing values before cleaning:
print(missing_before)

## # A tibble: 12 x 2
##   variable    missing_count
##   <chr>          <int>
## 1 brand              0
## 2 model              0
## 3 model_year         0
## 4 milage             0
## 5 fuel_type         170
## 6 engine             0
## 7 transmission       0
## 8 ext_col            0
## 9 int_col            0
## 10 accident         113
## 11 clean_title       596
## 12 price             0

# -----
# 2. Parse numeric variables

```

```

# -----

car_parsed <- car_raw %>%
  mutate(
    price_num = parse_number(price),
    mileage_num = parse_number(milage),
    model_year_num = as.integer(model_year),

    brand = str_squish(as.character(brand)),
    model = str_squish(as.character(model)),
    fuel_type = str_squish(as.character(fuel_type)),
    transmission = str_squish(as.character(transmission)),
    accident = str_squish(as.character(accident)),
    clean_title = str_squish(as.character(clean_title))
  )

# -----
# 3. Remove impossible or extreme rows
# -----
# price <= 200000 removes very high-end exotic listings.
# If you want to keep exotic cars, set USE_MAINSTREAM_FILTER <- FALSE.

USE_MAINSTREAM_FILTER <- TRUE
MAX_PRICE <- 200000
MAX_MILEAGE <- 300000
MIN_MODEL_YEAR <- 1980

car_filtered <- car_parsed %>%
  filter(
    !is.na(price_num),
    !is.na(mileage_num),
    !is.na(model_year_num),
    !is.na(brand),
    price_num > 0,
    mileage_num >= 0,
    model_year_num >= MIN_MODEL_YEAR,
    model_year_num <= max(model_year_num, na.rm = TRUE)
  )

if (USE_MAINSTREAM_FILTER) {
  car_filtered <- car_filtered %>%
    filter(
      price_num <= MAX_PRICE,
      mileage_num <= MAX_MILEAGE
    )
}

cat("\nRows after numeric parsing and filtering:\n")

##
## Rows after numeric parsing and filtering:

```

```

print(nrow(car_filtered))

## [1] 3928
cat("Rows removed from raw data:\n")

## Rows removed from raw data:
print(nrow(car_raw) - nrow(car_filtered))

## [1] 81
# -----
# 4. Feature engineering
# -----

REFERENCE_YEAR <- max(car_filtered$model_year_num, na.rm = TRUE)

car_features <- car_filtered %>%
  mutate(
    age = REFERENCE_YEAR - model_year_num,
    log_price = log(price_num),
    log_mileage = log(mileage_num + 1)
  )

cat("\nReference year used to define vehicle age:\n")

##
## Reference year used to define vehicle age:
print(REFERENCE_YEAR)

## [1] 2024
# -----
# 5. Recode categorical variables
# -----

car_recode <- car_features %>%
  mutate(
    fuel_type_clean = case_when(
      is.na(fuel_type) ~ "Unknown/Other",
      fuel_type %in% c("-", "--", "not supported", "Not Supported") ~ "Unknown/Other",
      TRUE ~ fuel_type
    ),

    transmission_clean = case_when(
      is.na(transmission) ~ "Unknown",
      str_detect(str_to_lower(transmission), "manual|m/t") ~ "Manual",
      str_detect(str_to_lower(transmission), "cvt") ~ "CVT",
      str_detect(str_to_lower(transmission), "automatic|a/t|dual shift|auto") ~ "Automatic",
      transmission %in% c("-", "--") ~ "Unknown",
      TRUE ~ "Other"
    ),

    accident_clean = case_when(
      is.na(accident) ~ "Unknown",

```

```

    str_detect(str_to_lower(accident), "at least|accident|damage") ~ "Accident/damage reported",
    str_detect(str_to_lower(accident), "none") ~ "None reported",
    TRUE ~ "Unknown"
  ),

  clean_title_clean = case_when(
    is.na(clean_title) ~ "Unknown",
    clean_title %in% c("Yes", "yes", "Y", "y") ~ "Yes",
    TRUE ~ "Unknown"
  )
)

# -----
# 6. Collapse rare brands
# -----
# Keep brands with at least 20 listings.
# Combine smaller brands into "Other".

BRAND_MIN_N <- 20

brand_counts <- car_recode %>%
  count(brand, sort = TRUE)

brands_keep <- brand_counts %>%
  filter(n >= BRAND_MIN_N) %>%
  pull(brand)

car_model <- car_recode %>%
  mutate(
    brand_group = if_else(brand %in% brands_keep, brand, "Other")
  )

cat("\nNumber of original brand levels:\n")

##
## Number of original brand levels:
print(n_distinct(car_recode$brand))

## [1] 56
cat("Number of brand levels after combining rare brands:\n")

## Number of brand levels after combining rare brands:
print(n_distinct(car_model$brand_group))

## [1] 35
cat("\nBrand counts after combining rare brands:\n")

##
## Brand counts after combining rare brands:
print(car_model %>% count(brand_group, sort = TRUE))

## # A tibble: 35 x 2

```

```
##      brand_group      n
##      <chr>      <int>
## 1 Ford      382
## 2 BMW      375
## 3 Mercedes-Benz 306
## 4 Chevrolet 292
## 5 Audi      200
## 6 Toyota    199
## 7 Porsche   188
## 8 Lexus     163
## 9 Jeep      143
## 10 Land     130
## # i 25 more rows
```

```
# -----
# 7. Keep final modeling variables
# -----

car_model <- car_model %>%
  select(
    price = price_num,
    log_price,
    model_year = model_year_num,
    age,
    mileage = mileage_num,
    log_mileage,
    brand,
    brand_group,
    fuel_type = fuel_type_clean,
    transmission = transmission_clean,
    accident = accident_clean,
    clean_title = clean_title_clean
  ) %>%
  mutate(
    brand_group = factor(brand_group),
    fuel_type = factor(fuel_type),
    transmission = factor(transmission),
    accident = factor(accident),
    clean_title = factor(clean_title)
  )

cat("\nFinal cleaned data dimensions:\n")
```

```
##
## Final cleaned data dimensions:
```

```
print(dim(car_model))
```

```
## [1] 3928 12
```

```
cat("\nFinal numeric summaries:\n")
```

```
##
## Final numeric summaries:
```

```
print(summary(car_model %>% select(price, log_price, age, mileage, log_mileage)))
```

```
##      price      log_price      age      mileage
## Min.   : 2000   Min.   : 7.601   Min.   : 0.000   Min.   : 100
## 1st Qu.: 17000  1st Qu.: 9.741   1st Qu.: 4.000   1st Qu.: 24263
## Median : 30500  Median :10.325   Median : 7.000   Median : 54000
## Mean   : 37896  Mean   :10.258   Mean   : 8.526   Mean   : 65458
## 3rd Qu.: 48000  3rd Qu.:10.779   3rd Qu.:12.000   3rd Qu.: 95000
## Max.   :200000  Max.   :12.206   Max.   :32.000   Max.   :300000
## log_mileage
## Min.   : 4.615
## 1st Qu.:10.097
## Median :10.897
## Mean   :10.646
## 3rd Qu.:11.462
## Max.   :12.612
```

```
cat("\nFinal categorical level counts:\n")
```

```
##
## Final categorical level counts:
```

```
cat("brand_group levels:", nlevels(car_model$brand_group), "\n")
```

```
## brand_group levels: 35
```

```
cat("fuel_type levels:", nlevels(car_model$fuel_type), "\n")
```

```
## fuel_type levels: 6
```

```
cat("transmission levels:", nlevels(car_model$transmission), "\n")
```

```
## transmission levels: 5
```

```
cat("accident levels:", nlevels(car_model$accident), "\n")
```

```
## accident levels: 3
```

```
cat("clean_title levels:", nlevels(car_model$clean_title), "\n")
```

```
## clean_title levels: 2
```

```
level_summary <- tibble(
  variable = c("brand_group", "fuel_type", "transmission", "accident", "clean_title"),
  n_levels = c(
    nlevels(car_model$brand_group),
    nlevels(car_model$fuel_type),
    nlevels(car_model$transmission),
    nlevels(car_model$accident),
    nlevels(car_model$clean_title)
  )
)
```

```
cat("\nLevel summary table:\n")
```

```
##
## Level summary table:
```

```
print(level_summary)
```

```
## # A tibble: 5 x 2
##   variable      n_levels
```

```

##      <chr>           <int>
## 1 brand_group       35
## 2 fuel_type         6
## 3 transmission     5
## 4 accident         3
## 5 clean_title      2

# -----
# 8. Train/test split
# -----
# Stratified by brand_group so each brand group appears in both train and test.

set.seed(431)

car_model <- car_model %>%
  mutate(row_id = row_number())

train_ids <- car_model %>%
  group_by(brand_group) %>%
  sample_frac(size = 0.80) %>%
  ungroup() %>%
  pull(row_id)

train_data <- car_model %>%
  filter(row_id %in% train_ids)

test_data <- car_model %>%
  filter(!(row_id %in% train_ids))

cat("\nTraining and testing sample sizes:\n")

##
## Training and testing sample sizes:
cat("Train n =", nrow(train_data), "\n")

## Train n = 3143
cat("Test n  =", nrow(test_data), "\n")

## Test n  = 785

# -----
# 9. Standardize continuous variables
# -----
# Important: compute mean/sd from training data only.

age_mean <- mean(train_data$age)
age_sd <- sd(train_data$age)
log_mileage_mean <- mean(train_data$log_mileage)
log_mileage_sd <- sd(train_data$log_mileage)
y_mean <- mean(train_data$log_price)

train_data <- train_data %>%
  mutate(
    z_age = (age - age_mean) / age_sd,
    z_log_mileage = (log_mileage - log_mileage_mean) / log_mileage_sd,

```



```

    y = log_price - y_mean
  )

test_data <- test_data %>%
  mutate(
    z_age = (age - age_mean) / age_sd,
    z_log_mileage = (log_mileage - log_mileage_mean) / log_mileage_sd,
    y = log_price - y_mean
  )

scaling_info <- tibble(
  parameter = c("age_mean", "age_sd", "log_mileage_mean", "log_mileage_sd", "y_mean"),
  value = c(age_mean, age_sd, log_mileage_mean, log_mileage_sd, y_mean)
)

cat("\nScaling constants:\n")

##
## Scaling constants:
print(scaling_info)

## # A tibble: 5 x 2
##   parameter      value
##   <chr>          <dbl>
## 1 age_mean        8.53
## 2 age_sd          6.06
## 3 log_mileage_mean 10.7
## 4 log_mileage_sd   1.13
## 5 y_mean         10.3

# -----
# 10. Prepare model matrix and brand indices for JAGS
# -----

brand_levels <- levels(train_data$brand_group)
J <- length(brand_levels)

train_data <- train_data %>%
  mutate(brand_id = as.integer(factor(brand_group, levels = brand_levels)))

test_data <- test_data %>%
  mutate(brand_id = as.integer(factor(brand_group, levels = brand_levels)))

x_formula <- ~ z_age + z_log_mileage + fuel_type + transmission + accident + clean_title

X_train <- model.matrix(x_formula, data = train_data)
X_train <- X_train[, colnames(X_train) != "(Intercept)", drop = FALSE]

X_test_raw <- model.matrix(x_formula, data = test_data)
X_test_raw <- X_test_raw[, colnames(X_test_raw) != "(Intercept)", drop = FALSE]

align_matrix <- function(X, target_colnames) {
  X_aligned <- matrix(0, nrow = nrow(X), ncol = length(target_colnames))
  colnames(X_aligned) <- target_colnames

```

```

common_cols <- intersect(colnames(X), target_colnames)
X_aligned[, common_cols] <- X[, common_cols, drop = FALSE]
return(X_aligned)
}

X_test <- align_matrix(X_test_raw, colnames(X_train))

jags_data <- list(
  n = nrow(train_data),
  p = ncol(X_train),
  J = J,
  y = train_data$y,
  X = X_train,
  brand_id = train_data$brand_id
)

cat("\nJAGS data dimensions:\n")

##
## JAGS data dimensions:
cat("n =", jags_data$n, "\n")

## n = 3143
cat("p =", jags_data$p, "\n")

## p = 14
cat("J =", jags_data$J, "\n")

## J = 35

# -----
# 11. Save cleaned outputs
# -----

write_csv(car_model, "used_cars_cleaned_all.csv")
write_csv(train_data, "used_cars_train_cleaned.csv")
write_csv(test_data, "used_cars_test_cleaned.csv")
write_csv(level_summary, "used_cars_level_summary.csv")
write_csv(scaling_info, "used_cars_scaling_info.csv")

saveRDS(
  list(
    train_data = train_data,
    test_data = test_data,
    X_train = X_train,
    X_test = X_test,
    jags_data = jags_data,
    brand_levels = brand_levels,
    scaling_info = scaling_info,
    x_formula = x_formula
  ),
  "used_cars_model_inputs.rds"
)

```

```

cat("\nSaved files:\n")

##
## Saved files:
cat("- used_cars_cleaned_all.csv\n")

## - used_cars_cleaned_all.csv
cat("- used_cars_train_cleaned.csv\n")

## - used_cars_train_cleaned.csv
cat("- used_cars_test_cleaned.csv\n")

## - used_cars_test_cleaned.csv
cat("- used_cars_level_summary.csv\n")

## - used_cars_level_summary.csv
cat("- used_cars_scaling_info.csv\n")

## - used_cars_scaling_info.csv
cat("- used_cars_model_inputs.rds\n")

## - used_cars_model_inputs.rds
# -----
# 12. EDA summary figure (Section 2)
# -----

library(patchwork)

top_brand_groups <- car_model %>%
  count(brand_group, sort = TRUE) %>%
  slice_head(n = 10) %>%
  pull(brand_group)

eda_brands <- car_model %>%
  filter(brand_group %in% c(top_brand_groups, "Other")) %>%
  count(brand_group, sort = TRUE) %>%
  mutate(brand_group = fct_reorder(brand_group, n, .desc = TRUE))

p_log_price <- ggplot(car_model, aes(x = log_price)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white", alpha = 0.85) +
  labs(
    x = "log(price)",
    y = "Count",
    title = "A. Listing price (log scale)"
  ) +
  theme_minimal(base_size = 11)

eda_predictors <- car_model %>%
  select(age, log_mileage) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "value") %>%
  mutate(

```

```

    variable = recode(
      variable,
      age = "Age (years)",
      log_mileage = "log(mileage + 1)"
    )
  )
)

p_age_mileage <- ggplot(eda_predictors, aes(x = value)) +
  geom_histogram(bins = 30, fill = "gray50", color = "white", alpha = 0.85) +
  facet_wrap(~variable, scales = "free_x", nrow = 1) +
  labs(x = NULL, y = "Count", title = "B. Age and mileage") +
  theme_minimal(base_size = 11) +
  theme(strip.text = element_text(face = "bold"))

p_brand_counts <- ggplot(eda_brands, aes(x = n, y = brand_group)) +
  geom_col(fill = "gray30") +
  labs(
    x = "Listings",
    y = NULL,
    title = "C. Top brand groups (plus Other)"
  ) +
  theme_minimal(base_size = 11)

eda_figure <- (p_log_price | p_age_mileage) / p_brand_counts +
  plot_layout(heights = c(1, 1.05), widths = c(0.95, 1.25))

ggsave(
  "used_cars_eda_summary.pdf",
  eda_figure,
  width = 11,
  height = 8
)

ggsave(
  "used_cars_eda_summary.png",
  eda_figure,
  width = 11,
  height = 8,
  dpi = 300
)

cat("\nSaved EDA figure:\n")

##
## Saved EDA figure:
cat("- used_cars_eda_summary.pdf\n")

## - used_cars_eda_summary.pdf
cat("- used_cars_eda_summary.png\n")

## - used_cars_eda_summary.png
# -----
# 13. Final checks
# -----

```

```

cat("\nFinal checks:\n")

##
## Final checks:
cat("Any missing values in final model data? ", any(is.na(car_model)), "\n")

## Any missing values in final model data? FALSE
cat("Any missing values in training data? ", any(is.na(train_data)), "\n")

## Any missing values in training data? FALSE
cat("Any missing values in testing data? ", any(is.na(test_data)), "\n")

## Any missing values in testing data? FALSE
cat("All test brands seen in training? ", all(test_data$brand_group %in% train_data$brand_group), "\n")

## All test brands seen in training? TRUE
cat("Number of fixed-effect columns in X_train:", ncol(X_train), "\n")

## Number of fixed-effect columns in X_train: 14
cat("Number of fixed-effect columns in X_test:", ncol(X_test), "\n")

## Number of fixed-effect columns in X_test: 14
# =====
# Bayesian Hierarchical Model for Used Car Price Prediction
# Step 2: Fit model using JAGS
# =====

library(tidyverse)
library(rjags)

## Loading required package: coda
## Linked to JAGS 4.3.2
## Loaded modules: basemod,bugs
library(coda)

# -----
# 1. Load cleaned model inputs
# -----

inputs <- readRDS("used_cars_model_inputs.rds")

train_data <- inputs$train_data
test_data <- inputs$test_data
X_train <- inputs$X_train
X_test <- inputs$X_test
jags_data <- inputs$jags_data
brand_levels <- inputs$brand_levels
scaling_info <- inputs$scaling_info

cat("Training observations:", nrow(train_data), "\n")

```

```

## Training observations: 3143
cat("Testing observations:", nrow(test_data), "\n")

## Testing observations: 785
cat("Number of fixed-effect predictors p:", jags_data$p, "\n")

## Number of fixed-effect predictors p: 14
cat("Number of brand groups J:", jags_data$J, "\n")

## Number of brand groups J: 35
# -----
# 2. Write JAGS model file
# -----

model_string <- "
model {

  # -----
  # Likelihood
  # -----
  for (i in 1:n) {
    y[i] ~ dnorm(mu[i], tau)
    mu[i] <- alpha[brand_id[i]] + inprod(X[i,], beta[])
  }

  # -----
  # Fixed-effect priors
  # -----
  for (k in 1:p) {
    beta[k] ~ dnorm(0, 0.25)
  }

  # -----
  # Brand-level random intercepts
  # -----
  for (j in 1:J) {
    alpha[j] ~ dnorm(mu_alpha, tau_alpha)
  }

  # -----
  # Hyperpriors
  # -----
  mu_alpha ~ dnorm(0, 0.04)

  sigma ~ dnorm(0, 0.25) T(0,)
  sigma_alpha ~ dnorm(0, 0.25) T(0,)

  tau <- 1 / (sigma * sigma)
  tau_alpha <- 1 / (sigma_alpha * sigma_alpha)
}
"

writeLines(model_string, con = "used_car_hier_model.txt")

```

```

# -----
# 3. Initial values
# -----

set.seed(431)

init_function <- function() {
  list(
    beta = rnorm(jags_data$p, 0, 0.1),
    alpha = rnorm(jags_data$J, 0, 0.1),
    mu_alpha = rnorm(1, 0, 0.1),
    sigma = runif(1, 0.5, 1.5),
    sigma_alpha = runif(1, 0.2, 1.0)
  )
}

# -----
# 4. Create and adapt JAGS model
# -----

n_chains <- 3

jags_model <- jags.model(
  file = "used_car_hier_model.txt",
  data = jags_data,
  inits = init_function,
  n.chains = n_chains,
  n.adapt = 1000
)

## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 3143
##   Unobserved stochastic nodes: 52
##   Total graph size: 59714
##
## Initializing model

# -----
# 5. Burn-in
# -----

update(jags_model, n.iter = 5000)

# -----
# 6. Draw posterior samples
# -----

params_to_monitor <- c(
  "beta",
  "alpha",
  "mu_alpha",

```

```

"sigma",
"sigma_alpha"
)

samples <- coda.samples(
  model = jags_model,
  variable.names = params_to_monitor,
  n.iter = 10000,
  thin = 5
)

# -----
# 6b. DIC (requires compiled jags_model in memory)
# -----
dic_baseline <- dic.samples(jags_model, n.iter = 2000, thin = 5)
cat("\nDIC (baseline hierarchical model, training data):\n")

##
## DIC (baseline hierarchical model, training data):
print(dic_baseline)

## Mean deviance: 3711
## penalty 48.43
## Penalized deviance: 3760
saveRDS(dic_baseline, "used_car_dic_baseline.rds")

# -----
# 7. Posterior summary
# -----

summary_samples <- summary(samples)

cat("\nPosterior summary:\n")

##
## Posterior summary:
print(summary_samples)

##
## Iterations = 6005:16000
## Thinning interval = 5
## Number of chains = 3
## Sample size per chain = 2000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## alpha[1]  0.111243 0.080433 0.0010384      0.0029758
## alpha[2]  0.400594 0.064982 0.0008389      0.0029041
## alpha[3]  1.421854 0.111116 0.0014345      0.0032223
## alpha[4]  0.453580 0.062362 0.0008051      0.0031152
## alpha[5]  0.007352 0.102490 0.0013231      0.0029028

```



```

## alpha[6]      0.412762 0.073188 0.0009448      0.0029596
## alpha[7]      0.405472 0.062439 0.0008061      0.0029510
## alpha[8]     -0.141778 0.106548 0.0013755      0.0033650
## alpha[9]      0.256773 0.075111 0.0009697      0.0027502
## alpha[10]     0.320397 0.058327 0.0007530      0.0027853
## alpha[11]     0.151923 0.117687 0.0015193      0.0031940
## alpha[12]     0.417412 0.073829 0.0009531      0.0027723
## alpha[13]     0.254745 0.083061 0.0010723      0.0028445
## alpha[14]    -0.200655 0.080916 0.0010446      0.0031147
## alpha[15]     0.216866 0.086062 0.0011111      0.0032037
## alpha[16]     0.488769 0.088260 0.0011394      0.0029047
## alpha[17]     0.284032 0.068238 0.0008809      0.0028986
## alpha[18]    -0.060165 0.078443 0.0010127      0.0030460
## alpha[19]     0.607347 0.069762 0.0009006      0.0029820
## alpha[20]     0.440741 0.067507 0.0008715      0.0029924
## alpha[21]     0.281659 0.084843 0.0010953      0.0028473
## alpha[22]     0.468171 0.102436 0.0013224      0.0029138
## alpha[23]    -0.062471 0.084367 0.0010892      0.0029842
## alpha[24]     0.581442 0.060881 0.0007860      0.0029209
## alpha[25]    -0.257204 0.102880 0.0013282      0.0031578
## alpha[26]     0.146749 0.119318 0.0015404      0.0033982
## alpha[27]     0.135167 0.072521 0.0009362      0.0029659
## alpha[28]     0.550399 0.072354 0.0009341      0.0031379
## alpha[29]     0.998916 0.067297 0.0008688      0.0029683
## alpha[30]     0.436989 0.069314 0.0008948      0.0024352
## alpha[31]     0.150472 0.086715 0.0011195      0.0031739
## alpha[32]     0.526877 0.088604 0.0011439      0.0031422
## alpha[33]     0.294792 0.065774 0.0008491      0.0030592
## alpha[34]    -0.086132 0.084521 0.0010912      0.0028698
## alpha[35]     0.111781 0.096661 0.0012479      0.0031520
## beta[1]      -0.362989 0.011006 0.0001421      0.0001538
## beta[2]      -0.282418 0.010654 0.0001375      0.0001471
## beta[3]      -0.545695 0.062473 0.0008065      0.0025016
## beta[4]      -0.467330 0.048651 0.0006281      0.0024534
## beta[5]      -0.487373 0.061305 0.0007914      0.0023657
## beta[6]      -0.503614 0.105192 0.0013580      0.0024344
## beta[7]      -0.554171 0.065167 0.0008413      0.0026116
## beta[8]      -0.244022 0.054562 0.0007044      0.0006993
## beta[9]       0.161485 0.030287 0.0003910      0.0003960
## beta[10]     -0.073417 0.111007 0.0014331      0.0014763
## beta[11]     0.417884 0.308388 0.0039813      0.0040361
## beta[12]     0.091200 0.019068 0.0002462      0.0003506
## beta[13]     0.125082 0.057471 0.0007419      0.0010786
## beta[14]     0.008596 0.026404 0.0003409      0.0007007
## mu_alpha     0.300924 0.080861 0.0010439      0.0029199
## sigma        0.436797 0.005531 0.0000714      0.0000715
## sigma_alpha  0.345636 0.045848 0.0005919      0.0006015
##
## 2. Quantiles for each variable:
##
##           2.5%      25%      50%      75%      97.5%
## alpha[1]   -0.047408 0.05630 0.111198 0.164913 0.27050
## alpha[2]    0.274581 0.35735 0.399560 0.443362 0.52585
## alpha[3]    1.201155 1.34704 1.422565 1.494996 1.63920

```

```

## alpha[4]      0.334333  0.41149  0.452301  0.494642  0.58056
## alpha[5]     -0.196794 -0.06187  0.006909  0.074714  0.21029
## alpha[6]      0.273557  0.36204  0.412548  0.462264  0.55745
## alpha[7]      0.286354  0.36285  0.404157  0.446693  0.52980
## alpha[8]     -0.356651 -0.21386 -0.140934 -0.070051  0.06746
## alpha[9]      0.112658  0.20618  0.257044  0.307173  0.40352
## alpha[10]     0.207808  0.28027  0.319696  0.360360  0.43389
## alpha[11]    -0.079776  0.07354  0.153053  0.231428  0.37898
## alpha[12]     0.274137  0.36770  0.417396  0.468072  0.56209
## alpha[13]     0.093381  0.20018  0.253985  0.310391  0.42062
## alpha[14]    -0.354454 -0.25615 -0.201739 -0.145131 -0.04167
## alpha[15]     0.054003  0.15778  0.217358  0.274559  0.38776
## alpha[16]     0.319028  0.42825  0.487315  0.548190  0.66267
## alpha[17]     0.153351  0.23787  0.283407  0.329921  0.42246
## alpha[18]    -0.209471 -0.11353 -0.062264 -0.007855  0.09393
## alpha[19]     0.473612  0.55830  0.606977  0.654161  0.74520
## alpha[20]     0.311234  0.39457  0.438732  0.486458  0.57514
## alpha[21]     0.115782  0.22351  0.281721  0.337935  0.45103
## alpha[22]     0.271837  0.39876  0.468062  0.535654  0.67226
## alpha[23]    -0.225139 -0.11943 -0.064410 -0.004142  0.10450
## alpha[24]     0.464539  0.53967  0.580379  0.624053  0.70089
## alpha[25]    -0.460594 -0.32789 -0.257745 -0.185739 -0.05523
## alpha[26]    -0.084059  0.06438  0.148656  0.226264  0.38003
## alpha[27]    -0.003038  0.08516  0.133938  0.184539  0.27694
## alpha[28]     0.409657  0.50134  0.548518  0.599144  0.69226
## alpha[29]     0.869929  0.95359  0.997399  1.044742  1.12858
## alpha[30]     0.303924  0.38877  0.436441  0.484008  0.57309
## alpha[31]    -0.016014  0.09167  0.149482  0.209999  0.31914
## alpha[32]     0.355636  0.46589  0.526358  0.587423  0.70193
## alpha[33]     0.170342  0.25057  0.294427  0.339365  0.42475
## alpha[34]    -0.249899 -0.14428 -0.086740 -0.028403  0.07860
## alpha[35]    -0.080409  0.04616  0.111664  0.175216  0.30509
## beta[1]      -0.384887 -0.37045 -0.363097 -0.355445 -0.34165
## beta[2]      -0.302883 -0.28962 -0.282375 -0.275193 -0.26143
## beta[3]      -0.670935 -0.58791 -0.545258 -0.503829 -0.42237
## beta[4]      -0.563359 -0.49905 -0.466750 -0.434300 -0.37313
## beta[5]      -0.608008 -0.52873 -0.486594 -0.446779 -0.36893
## beta[6]      -0.707244 -0.57460 -0.502927 -0.432906 -0.29870
## beta[7]      -0.681624 -0.59750 -0.554151 -0.509589 -0.42636
## beta[8]      -0.350090 -0.28144 -0.243761 -0.207356 -0.13740
## beta[9]       0.101287  0.14056  0.161022  0.182481  0.22084
## beta[10]     -0.289198 -0.15061 -0.072966  0.002102  0.14715
## beta[11]     -0.177249  0.20709  0.417567  0.626944  1.02400
## beta[12]     0.052919  0.07831  0.091371  0.103705  0.12849
## beta[13]     0.012352  0.08685  0.125307  0.162992  0.23638
## beta[14]    -0.043417 -0.00935  0.008801  0.026639  0.05991
## mu_alpha     0.144343  0.24485  0.301543  0.356211  0.45735
## sigma        0.426007  0.43300  0.436764  0.440500  0.44753
## sigma_alpha  0.269438  0.31362  0.340124  0.373145  0.44876

```

```

# -----
# 8. Convergence diagnostics
# -----

```

```

cat("\nGelman-Rubin diagnostics:\n")

##
## Gelman-Rubin diagnostics:
gelman_results <- gelman.diag(samples, multivariate = FALSE)
print(gelman_results)

## Potential scale reduction factors:
##
##          Point est. Upper C.I.
## alpha[1]          1      1.01
## alpha[2]          1      1.01
## alpha[3]          1      1.00
## alpha[4]          1      1.01
## alpha[5]          1      1.01
## alpha[6]          1      1.01
## alpha[7]          1      1.01
## alpha[8]          1      1.01
## alpha[9]          1      1.01
## alpha[10]         1      1.02
## alpha[11]         1      1.00
## alpha[12]         1      1.01
## alpha[13]         1      1.01
## alpha[14]         1      1.01
## alpha[15]         1      1.01
## alpha[16]         1      1.01
## alpha[17]         1      1.01
## alpha[18]         1      1.01
## alpha[19]         1      1.01
## alpha[20]         1      1.01
## alpha[21]         1      1.01
## alpha[22]         1      1.00
## alpha[23]         1      1.01
## alpha[24]         1      1.01
## alpha[25]         1      1.01
## alpha[26]         1      1.00
## alpha[27]         1      1.01
## alpha[28]         1      1.01
## alpha[29]         1      1.01
## alpha[30]         1      1.01
## alpha[31]         1      1.00
## alpha[32]         1      1.01
## alpha[33]         1      1.01
## alpha[34]         1      1.00
## alpha[35]         1      1.01
## beta[1]           1      1.00
## beta[2]           1      1.01
## beta[3]           1      1.01
## beta[4]           1      1.01
## beta[5]           1      1.01
## beta[6]           1      1.01
## beta[7]           1      1.01
## beta[8]           1      1.00

```

```

## beta[9]          1      1.00
## beta[10]         1      1.00
## beta[11]         1      1.00
## beta[12]         1      1.01
## beta[13]         1      1.00
## beta[14]         1      1.00
## mu_alpha         1      1.01
## sigma            1      1.00
## sigma_alpha      1      1.00

cat("\nEffective sample sizes:\n")

##
## Effective sample sizes:
ess_results <- effectiveSize(samples)
print(ess_results)

##      alpha[1]      alpha[2]      alpha[3]      alpha[4]      alpha[5]      alpha[6]
##      754.8385      532.1714      1335.7363      413.6529      1266.3204      630.9033
##      alpha[7]      alpha[8]      alpha[9]      alpha[10]     alpha[11]     alpha[12]
##      460.8028      1136.9994      810.9202      447.0692      1361.8061      739.2318
##      alpha[13]     alpha[14]     alpha[15]     alpha[16]     alpha[17]     alpha[18]
##      869.5764      733.0288      746.6068      975.0539      557.6693      713.7904
##      alpha[19]     alpha[20]     alpha[21]     alpha[22]     alpha[23]     alpha[24]
##      560.6109      522.6412      935.5212      1283.5799      828.6916      458.2316
##      alpha[25]     alpha[26]     alpha[27]     alpha[28]     alpha[29]     alpha[30]
##      1134.6192     1379.2484      637.4244      575.1439      536.3839      823.1609
##      alpha[31]     alpha[32]     alpha[33]     alpha[34]     alpha[35]      beta[1]
##      784.5760      805.0439      481.4667      889.9485      1036.4815     5128.5307
##      beta[2]       beta[3]       beta[4]       beta[5]       beta[6]       beta[7]
##      5255.2919     677.9434      402.0681      685.8334      1870.1577     661.8046
##      beta[8]       beta[9]       beta[10]      beta[11]      beta[12]      beta[13]
##      6090.0830     5858.1650     5683.4528     5853.1918     2969.9175     2865.4263
##      beta[14]      mu_alpha      sigma      sigma_alpha
##      1421.6338      811.7294     5985.7131     5826.0372

# -----
# 8b. Monte Carlo standard errors (baseline model)
# -----
# Key estimands: residual scale (sigma), brand-level spread (sigma_alpha),
# overall brand intercept (mu_alpha), and first fixed-effect betas.
mcse_targets <- c(
  "sigma",
  "sigma_alpha",
  "mu_alpha",
  paste0("beta[", seq_len(min(3, jags_data$p)), "]", ")")
)

sum_stats <- summary(samples)$statistics
mcse_results <- sum_stats[mcse_targets, "Time-series SE", drop = TRUE]
names(mcse_results) <- mcse_targets

cat("\nMonte Carlo standard errors (time-series SE from coda::summary):\n")

##

```

```
## Monte Carlo standard errors (time-series SE from coda::summary):
print(mcse_results)

##          sigma  sigma_alpha    mu_alpha    beta[1]    beta[2]    beta[3]
## 7.149707e-05 6.014974e-04 2.919877e-03 1.538089e-04 1.471173e-04 2.501591e-03

mcse_summary <- tibble(
  parameter = mcse_targets,
  posterior_mean = sum_stats[mcse_targets, "Mean"],
  posterior_sd = sum_stats[mcse_targets, "SD"],
  ess = as.numeric(ess_results[mcse_targets]),
  mcse = as.numeric(mcse_results),
) %>%
  mutate(mcse_over_sd = mcse / posterior_sd)

cat("\nMCSE summary table:\n")

##
## MCSE summary table:
print(mcse_summary)

## # A tibble: 6 x 6
##   parameter    posterior_mean posterior_sd    ess      mcse mcse_over_sd
##   <chr>          <dbl>          <dbl> <dbl>    <dbl>    <dbl>
## 1 sigma          0.437          0.00553 5986. 0.0000715 0.0129
## 2 sigma_alpha    0.346          0.0458  5826. 0.000601 0.0131
## 3 mu_alpha       0.301          0.0809   812. 0.00292 0.0361
## 4 beta[1]       -0.363          0.0110  5129. 0.000154 0.0140
## 5 beta[2]       -0.282          0.0107  5255. 0.000147 0.0138
## 6 beta[3]       -0.546          0.0625   678. 0.00250 0.0400

write_csv(mcse_summary, "mcmc_mcse_summary.csv")

# -----
# 9. Trace plots
# -----

# Trace plots for main variance/hyperparameters
pdf("traceplots_main_parameters.pdf", width = 10, height = 8)
traceplot(samples[, c("mu_alpha", "sigma", "sigma_alpha")])
dev.off()

## pdf
## 2

# Trace plots for first few beta coefficients
beta_names <- paste0("beta[", 1:min(6, jags_data$p), "]")

pdf("traceplots_beta_parameters.pdf", width = 10, height = 8)
traceplot(samples[, beta_names])
dev.off()

## pdf
## 2
```

```

# -----
# 10. Save model outputs
# -----

saveRDS(samples, "used_car_jags_samples.rds")
saveRDS(gelman_results, "used_car_gelman_results.rds")
saveRDS(ess_results, "used_car_effective_sample_sizes.rds")
saveRDS(summary_samples, "used_car_posterior_summary.rds")
saveRDS(mcse_results, "used_car_mcse_results.rds")

cat("\nSaved files:\n")

##
## Saved files:
cat("- used_car_hier_model.txt\n")

## - used_car_hier_model.txt
cat("- used_car_jags_samples.rds\n")

## - used_car_jags_samples.rds
cat("- used_car_dic_baseline.rds\n")

## - used_car_dic_baseline.rds
cat("- used_car_gelman_results.rds\n")

## - used_car_gelman_results.rds
cat("- used_car_effective_sample_sizes.rds\n")

## - used_car_effective_sample_sizes.rds
cat("- used_car_mcse_results.rds\n")

## - used_car_mcse_results.rds
cat("- mcmc_mcse_summary.csv\n")

## - mcmc_mcse_summary.csv
cat("- used_car_posterior_summary.rds\n")

## - used_car_posterior_summary.rds
cat("- traceplots_main_parameters.pdf\n")

## - traceplots_main_parameters.pdf
cat("- traceplots_beta_parameters.pdf\n")

## - traceplots_beta_parameters.pdf
# =====
# Prior sensitivity: alternative fit (tighter beta prior only)
# =====
# Baseline: beta[k] ~ dnorm(0, 0.25) => variance 4, SD = 2 (JAGS uses precision).
# Alternative: beta[k] ~ dnorm(0, 1) => variance 1, SD = 1 (more shrinkage toward 0).
# Same likelihood, same alpha/sigma priors, same jags_data - only pi(beta) changes.
# Use printed tables / CSV outputs for the report text (replace any draft placeholders).

```

```

library(tidyverse)
library(rjags)
library(coda)

inputs_ps <- readRDS("used_cars_model_inputs.rds")
jags_data_ps <- inputs_ps$jags_data

model_string_alt <- "
model {
  for (i in 1:n) {
    y[i] ~ dnorm(mu[i], tau)
    mu[i] <- alpha[brand_id[i]] + inprod(X[i,], beta[])
  }
  for (k in 1:p) {
    beta[k] ~ dnorm(0, 1)
  }
  for (j in 1:J) {
    alpha[j] ~ dnorm(mu_alpha, tau_alpha)
  }
  mu_alpha ~ dnorm(0, 0.04)
  sigma ~ dnorm(0, 0.25) T(0,)
  sigma_alpha ~ dnorm(0, 0.25) T(0,)
  tau <- 1 / (sigma * sigma)
  tau_alpha <- 1 / (sigma_alpha * sigma_alpha)
}
"

writeLines(model_string_alt, con = "used_car_hier_model_prior_alt.txt")

set.seed(431)

init_function_alt <- function() {
  list(
    beta = rnorm(jags_data_ps$p, 0, 0.1),
    alpha = rnorm(jags_data_ps$J, 0, 0.1),
    mu_alpha = rnorm(1, 0, 0.1),
    sigma = runif(1, 0.5, 1.5),
    sigma_alpha = runif(1, 0.2, 1.0)
  )
}

jags_model_alt <- jags.model(
  file = "used_car_hier_model_prior_alt.txt",
  data = jags_data_ps,
  inits = init_function_alt,
  n.chains = 3,
  n.adapt = 1000
)

## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:

```

```

## Observed stochastic nodes: 3143
## Unobserved stochastic nodes: 52
## Total graph size: 59714
##
## Initializing model
update(jags_model_alt, n.iter = 5000)

samples_alt <- coda.samples(
  model = jags_model_alt,
  variable.names = c("beta", "alpha", "mu_alpha", "sigma", "sigma_alpha"),
  n.iter = 10000,
  thin = 5
)

dic_alt <- dic.samples(jags_model_alt, n.iter = 2000, thin = 5)
cat("\nDIC (alternative beta prior, training data):\n")

##
## DIC (alternative beta prior, training data):
print(dic_alt)

## Mean deviance: 3711
## penalty 48.93
## Penalized deviance: 3760

saveRDS(dic_alt, "used_car_dic_prior_alt.rds")

saveRDS(samples_alt, "used_car_jags_samples_prior_alt.rds")

# --- Compare key posteriors: baseline vs alternative ---
m_base <- as.matrix(readRDS("used_car_jags_samples.rds"))
m_alt <- as.matrix(samples_alt)

summ_par <- function(mat, par) {
  c(mean = mean(mat[, par]), q025 = quantile(mat[, par], 0.025), q975 = quantile(mat[, par], 0.975))
}

prior_sens_compare <- tibble::tibble(
  parameter = c("sigma", "sigma_alpha", "mu_alpha", "beta[1]", "beta[2]", "beta[3]"),
  baseline_mean = c(
    mean(m_base[, "sigma"]),
    mean(m_base[, "sigma_alpha"]),
    mean(m_base[, "mu_alpha"]),
    mean(m_base[, "beta[1]"]),
    mean(m_base[, "beta[2]"]),
    mean(m_base[, "beta[3]"])
  ),
  alt_mean = c(
    mean(m_alt[, "sigma"]),
    mean(m_alt[, "sigma_alpha"]),
    mean(m_alt[, "mu_alpha"]),
    mean(m_alt[, "beta[1]"]),
    mean(m_alt[, "beta[2]"]),
    mean(m_alt[, "beta[3]"])
  )
)

```



```

)
) %>%
  mutate(diff = alt_mean - baseline_mean)

cat("\nPrior sensitivity: posterior mean comparison (alt - baseline beta prior)\n")

##
## Prior sensitivity: posterior mean comparison (alt - baseline beta prior)
print(prior_sens_compare)

## # A tibble: 6 x 4
##   parameter    baseline_mean alt_mean      diff
##   <chr>          <dbl>    <dbl>    <dbl>
## 1 sigma          0.437      0.437  0.0000869
## 2 sigma_alpha    0.346      0.345 -0.000232
## 3 mu_alpha       0.301      0.291 -0.0101
## 4 beta[1]       -0.363     -0.363 -0.000237
## 5 beta[2]       -0.282     -0.282  0.000195
## 6 beta[3]       -0.546     -0.536  0.00925

write_csv(prior_sens_compare, "prior_sensitivity_posterior_compare.csv")

# --- Test RMSE / MAE under alternative (same test set, same prediction code as Step 3) ---
test_data_ps <- inputs_ps$test_data
X_test_ps <- inputs_ps$X_test
scaling_info_ps <- inputs_ps$scaling_info
brand_levels_ps <- inputs_ps$brand_levels
y_mean_ps <- scaling_info_ps$value[scaling_info_ps$parameter == "y_mean"]

p_ps <- ncol(X_test_ps)
J_ps <- length(brand_levels_ps)
beta_cols_ps <- paste0("beta[", 1:p_ps, "]")
alpha_cols_ps <- paste0("alpha[", 1:J_ps, "]")

beta_alt <- m_alt[, beta_cols_ps, drop = FALSE]
alpha_alt <- m_alt[, alpha_cols_ps, drop = FALSE]
sigma_alt <- m_alt[, "sigma"]

n_test_ps <- nrow(test_data_ps)
n_draws_alt <- nrow(m_alt)
pred_log_alt <- matrix(NA, nrow = n_draws_alt, ncol = n_test_ps)

for (s in seq_len(n_draws_alt)) {
  fixed_part <- as.vector(X_test_ps %*% beta_alt[s, ])
  brand_part <- alpha_alt[s, test_data_ps$brand_id]
  mu_log_price <- fixed_part + brand_part + y_mean_ps
  pred_log_alt[s, ] <- rnorm(n_test_ps, mean = mu_log_price, sd = sigma_alt[s])
}

pred_price_alt <- exp(pred_log_alt)
bayes_pred_price_alt <- colMeans(pred_price_alt)
actual_ps <- test_data_ps$price
rmse_ps <- function(a, p) sqrt(mean((a - p)^2))
mae_ps <- function(a, p) mean(abs(a - p))

```

```

bayes_rmse_alt <- rmse_ps(actual_ps, bayes_pred_price_alt)
bayes_mae_alt <- mae_ps(actual_ps, bayes_pred_price_alt)

# Reload baseline test metrics from same procedure (recompute from baseline samples for fair side-by-side)
beta_base <- m_base[, beta_cols_ps, drop = FALSE]
alpha_base <- m_base[, alpha_cols_ps, drop = FALSE]
sigma_base_vec <- m_base[, "sigma"]
n_draws_base <- nrow(m_base)
pred_log_base <- matrix(NA, nrow = n_draws_base, ncol = n_test_ps)
for (s in seq_len(n_draws_base)) {
  fixed_part <- as.vector(X_test_ps %*% beta_base[s, ])
  brand_part <- alpha_base[s, test_data_ps$brand_id]
  mu_log_price <- fixed_part + brand_part + y_mean_ps
  pred_log_base[s, ] <- rnorm(n_test_ps, mean = mu_log_price, sd = sigma_base_vec[s])
}
bayes_pred_price_base <- colMeans(exp(pred_log_base))
bayes_rmse_base <- rmse_ps(actual_ps, bayes_pred_price_base)
bayes_mae_base <- mae_ps(actual_ps, bayes_pred_price_base)

pred_metric_compare <- tibble::tibble(
  prior = c("beta ~ N(0, 2^2) baseline", "beta ~ N(0, 1^2) alternative"),
  test_RMSE = c(bayes_rmse_base, bayes_rmse_alt),
  test_MAE = c(bayes_mae_base, bayes_mae_alt)
)

cat("\nPrior sensitivity: test-set prediction metrics\n")

```

```

##
## Prior sensitivity: test-set prediction metrics

```

```

print(pred_metric_compare)

```

```

## # A tibble: 2 x 3
##   prior                test_RMSE test_MAE
##   <chr>                <dbl>    <dbl>
## 1 beta ~ N(0, 2^2) baseline    24118.    13442.
## 2 beta ~ N(0, 1^2) alternative    24029.    13430.

```

```

write_csv(pred_metric_compare, "prior_sensitivity_test_metrics.csv")

```

```

# =====
# WAIC / LOO-CV from pointwise log-likelihood (training data only)
# =====
# log_lik[s, i] = log p(y_i | theta^(s)) under the hierarchical normal model.
# Built in R from posterior draws (avoids monitoring n loglik nodes in JAGS).
# Compare baseline vs alternative prior on beta; DIC computed in JAGS chunks above.

```

```

library(tidyverse)
library(loo)

```

```

## Warning: package 'loo' was built under R version 4.4.3
## This is loo version 2.9.0
## - Online documentation and vignettes at mc-stan.org/loo

```

- As of v2.0.0 loo defaults to 1 core but we recommend using as many as possible. Use the 'cores' arg

```
inputs_ic <- readRDS("used_cars_model_inputs.rds")
jags_ic <- inputs_ic$jags_data
train_ic <- inputs_ic$train_data
Xtr <- as.matrix(inputs_ic$X_train)
y_tr <- jags_ic$y
brand_id <- train_ic$brand_id
p_ic <- jags_ic$p
J_ic <- jags_ic$J

build_log_lik <- function(m, y, X, brand_id, p, J) {
  S <- nrow(m)
  n <- length(y)
  beta_cols <- paste0("beta[", seq_len(p), "]")
  alpha_cols <- paste0("alpha[", seq_len(J), "]")
  beta_s <- m[, beta_cols, drop = FALSE]
  alpha_s <- m[, alpha_cols, drop = FALSE]
  sig <- m[, "sigma"]
  log_lik <- matrix(NA_real_, nrow = S, ncol = n)
  for (s in seq_len(S)) {
    mu <- alpha_s[s, brand_id] + as.numeric(X %*% beta_s[s, ])
    log_lik[s, ] <- stats::dnorm(y, mean = mu, sd = sig[s], log = TRUE)
  }
  log_lik
}

m_base <- as.matrix(readRDS("used_car_jags_samples.rds"))
m_alt <- as.matrix(readRDS("used_car_jags_samples_prior_alt.rds"))

log_lik_base <- build_log_lik(m_base, y_tr, Xtr, brand_id, p_ic, J_ic)
log_lik_alt <- build_log_lik(m_alt, y_tr, Xtr, brand_id, p_ic, J_ic)

waic_base <- waic(log_lik_base)
```

Warning:

6 (0.2%) p_waic estimates greater than 0.4. We recommend trying loo instead.

```
waic_alt <- waic(log_lik_alt)
```

Warning:

6 (0.2%) p_waic estimates greater than 0.4. We recommend trying loo instead.

```
loo_base <- loo(log_lik_base, save_psis = TRUE)
```

```
loo_alt <- loo(log_lik_alt, save_psis = TRUE)
```

```
cat("\nWAIC - baseline:\n")
```

##

WAIC - baseline:

```
print(waic_base)
```

##

Computed from 6000 by 3143 log-likelihood matrix.

##

Estimate SE

```
## elpd_waic -1880.0 50.4
## p_waic      47.2 2.1
## waic        3760.0 100.7
##
## 6 (0.2%) p_waic estimates greater than 0.4. We recommend trying loo instead.
```

```
cat("\nWAIC - alternative beta prior:\n")
```

```
##
## WAIC - alternative beta prior:
```

```
print(waic_alt)
```

```
##
## Computed from 6000 by 3143 log-likelihood matrix.
##
##           Estimate      SE
## elpd_waic -1879.8 50.3
## p_waic      46.9 2.0
## waic        3759.6 100.7
##
## 6 (0.2%) p_waic estimates greater than 0.4. We recommend trying loo instead.
```

```
cat("\nL00 - baseline:\n")
```

```
##
## L00 - baseline:
```

```
print(loo_base)
```

```
##
## Computed from 6000 by 3143 log-likelihood matrix.
##
##           Estimate      SE
## elpd_loo -1880.1 50.4
## p_loo      47.3 2.1
## looic      3760.2 100.7
## -----
## MCSE of elpd_loo is 0.1.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.
```

```
cat("\nL00 - alternative beta prior:\n")
```

```
##
## L00 - alternative beta prior:
```

```
print(loo_alt)
```

```
##
## Computed from 6000 by 3143 log-likelihood matrix.
##
##           Estimate      SE
## elpd_loo -1879.9 50.3
## p_loo      47.0 2.0
## looic      3759.9 100.7
```

```

## -----
## MCSE of elpd_loo is 0.1.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.

loo_comp <- loo_compare(list(baseline = loo_base, alternative_beta_prior = loo_alt))
cat("\nloo_compare (sorted by elpd; first row is best; elpd_diff is vs. that row):\n")

##
## loo_compare (sorted by elpd; first row is best; elpd_diff is vs. that row):
print(loo_comp)

##
##          elpd_diff se_diff
## alternative_beta_prior 0.0      0.0
## baseline               -0.2     0.2

saveRDS(loo_base, "used_car_loo_baseline.rds")
saveRDS(loo_alt, "used_car_loo_prior_alt.rds")
saveRDS(waic_base, "used_car_waic_baseline.rds")
saveRDS(waic_alt, "used_car_waic_prior_alt.rds")

# --- Flat tables for the report ---
dic_b <- readRDS("used_car_dic_baseline.rds")
dic_a <- readRDS("used_car_dic_prior_alt.rds")

dic_tbl <- tibble(
  model = c("baseline_beta_N02", "alternative_beta_N01"),
  mean_deviance = c(mean(as.matrix(dic_b$deviance)), mean(as.matrix(dic_a$deviance))),
  penalty_pD = c(mean(as.matrix(dic_b$penalty)), mean(as.matrix(dic_a$penalty))),
  DIC = mean_deviance + penalty_pD
)

waic_tbl <- tibble(
  model = c("baseline_beta_N02", "alternative_beta_N01"),
  waic = c(waic_base$estimates["waic", "Estimate"], waic_alt$estimates["waic", "Estimate"]),
  se = c(waic_base$estimates["waic", "SE"], waic_alt$estimates["waic", "SE"])
)

loo_tbl <- tibble(
  model = c("baseline_beta_N02", "alternative_beta_N01"),
  elpd_loo = c(loo_base$estimates["elpd_loo", "Estimate"], loo_alt$estimates["elpd_loo", "Estimate"]),
  se_elpd = c(loo_base$estimates["elpd_loo", "SE"], loo_alt$estimates["elpd_loo", "SE"]),
  looic = c(loo_base$estimates["looic", "Estimate"], loo_alt$estimates["looic", "Estimate"]),
  p_loo = c(loo_base$estimates["p_loo", "Estimate"], loo_alt$estimates["p_loo", "Estimate"])
)

loo_comp_df <- tibble::rownames_to_column(as.data.frame(as.matrix(loo_comp)), "model") %>%
  as_tibble()

write_csv(dic_tbl, "model_comparison_dic.csv")
write_csv(waic_tbl, "model_comparison_waic.csv")
write_csv(loo_tbl, "model_comparison_loo.csv")
write_csv(loo_comp_df, "model_comparison_loo_compare.csv")

```

```

cat("\nSaved: model_comparison_dic.csv, model_comparison_waic.csv,\n")

##
## Saved: model_comparison_dic.csv, model_comparison_waic.csv,
cat("      model_comparison_loo.csv, model_comparison_loo_compare.csv\n")

##      model_comparison_loo.csv, model_comparison_loo_compare.csv
gelman_results <- readRDS("used_car_gelman_results.rds")
print(gelman_results)

## Potential scale reduction factors:
##
##      Point est. Upper C.I.
## alpha[1]      1      1.01
## alpha[2]      1      1.01
## alpha[3]      1      1.00
## alpha[4]      1      1.01
## alpha[5]      1      1.01
## alpha[6]      1      1.01
## alpha[7]      1      1.01
## alpha[8]      1      1.01
## alpha[9]      1      1.01
## alpha[10]     1      1.02
## alpha[11]     1      1.00
## alpha[12]     1      1.01
## alpha[13]     1      1.01
## alpha[14]     1      1.01
## alpha[15]     1      1.01
## alpha[16]     1      1.01
## alpha[17]     1      1.01
## alpha[18]     1      1.01
## alpha[19]     1      1.01
## alpha[20]     1      1.01
## alpha[21]     1      1.01
## alpha[22]     1      1.00
## alpha[23]     1      1.01
## alpha[24]     1      1.01
## alpha[25]     1      1.01
## alpha[26]     1      1.00
## alpha[27]     1      1.01
## alpha[28]     1      1.01
## alpha[29]     1      1.01
## alpha[30]     1      1.01
## alpha[31]     1      1.00
## alpha[32]     1      1.01
## alpha[33]     1      1.01
## alpha[34]     1      1.00
## alpha[35]     1      1.01
## beta[1]       1      1.00
## beta[2]       1      1.01
## beta[3]       1      1.01
## beta[4]       1      1.01
## beta[5]       1      1.01

```

```

## beta[6]          1      1.01
## beta[7]          1      1.01
## beta[8]          1      1.00
## beta[9]          1      1.00
## beta[10]         1      1.00
## beta[11]         1      1.00
## beta[12]         1      1.01
## beta[13]         1      1.00
## beta[14]         1      1.00
## mu_alpha         1      1.01
## sigma            1      1.00
## sigma_alpha      1      1.00

# =====
# Step 3: Posterior prediction and OLS comparison
# =====

library(tidyverse)
library(coda)

# -----
# 1. Load saved objects
# -----

inputs <- readRDS("used_cars_model_inputs.rds")
samples <- readRDS("used_car_jags_samples.rds")

train_data <- inputs$train_data
test_data <- inputs$test_data
X_train <- inputs$X_train
X_test <- inputs$X_test
scaling_info <- inputs$scaling_info
brand_levels <- inputs$brand_levels

# y_mean was used to center log_price during cleaning
y_mean <- scaling_info$value[scaling_info$parameter == "y_mean"]

# Convert MCMC samples to matrix
sample_matrix <- as.matrix(samples)

cat("Number of posterior draws:", nrow(sample_matrix), "\n")

## Number of posterior draws: 6000

cat("Number of test observations:", nrow(test_data), "\n")

## Number of test observations: 785

# -----
# 2. Extract posterior samples
# -----

p <- ncol(X_test)
J <- length(brand_levels)

beta_cols <- paste0("beta[", 1:p, "]")

```

```

alpha_cols <- paste0("alpha[", 1:J, "]")

beta_samples <- sample_matrix[, beta_cols, drop = FALSE]
alpha_samples <- sample_matrix[, alpha_cols, drop = FALSE]
sigma_samples <- sample_matrix[, "sigma"]

# -----
# 3. Posterior predictive mean for test set
# -----

n_test <- nrow(test_data)
n_draws <- nrow(sample_matrix)

# To save memory, we calculate predictive means directly
pred_log_price_draws <- matrix(NA, nrow = n_draws, ncol = n_test)

for (s in 1:n_draws) {
  fixed_part <- as.vector(X_test %*% beta_samples[s, ])
  brand_part <- alpha_samples[s, test_data$brand_id]
  mu_centered <- brand_part + fixed_part

  # Add back y_mean to return to log(price) scale
  mu_log_price <- mu_centered + y_mean

  # Optional: generate posterior predictive samples including residual noise
  pred_log_price_draws[s, ] <- rnorm(
    n = n_test,
    mean = mu_log_price,
    sd = sigma_samples[s]
  )
}

# Convert back to dollar price scale
pred_price_draws <- exp(pred_log_price_draws)

# Posterior predictive mean
bayes_pred_price <- colMeans(pred_price_draws)

# 95% posterior predictive intervals
bayes_pred_lower <- apply(pred_price_draws, 2, quantile, probs = 0.025)
bayes_pred_upper <- apply(pred_price_draws, 2, quantile, probs = 0.975)

# -----
# 4. Bayesian model prediction metrics
# -----

actual_price <- test_data$price

rmse <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2))
}

mae <- function(actual, predicted) {

```



```

    mean(abs(actual - predicted))
  }

bayes_rmse <- rmse(actual_price, bayes_pred_price)
bayes_mae <- mae(actual_price, bayes_pred_price)

cat("\nBayesian hierarchical model performance:\n")

##
## Bayesian hierarchical model performance:
cat("RMSE:", bayes_rmse, "\n")

## RMSE: 24098.7
cat("MAE :", bayes_mae, "\n")

## MAE : 13441
# Prediction interval coverage
coverage_95 <- mean(actual_price >= bayes_pred_lower & actual_price <= bayes_pred_upper)

cat("95% predictive interval coverage:", coverage_95, "\n")

## 95% predictive interval coverage: 0.9426752
# -----
# 5. OLS baseline model
# -----
# Use same fixed predictors plus brand_group as ordinary regression dummies.

ols_model <- lm(
  log_price ~ z_age + z_log_mileage +
    fuel_type + transmission + accident + clean_title + brand_group,
  data = train_data
)

ols_pred_log_price <- predict(ols_model, newdata = test_data)

# Convert log prediction back to price scale
ols_pred_price <- exp(ols_pred_log_price)

ols_rmse <- rmse(actual_price, ols_pred_price)
ols_mae <- mae(actual_price, ols_pred_price)

cat("\nOLS model performance:\n")

##
## OLS model performance:
cat("RMSE:", ols_rmse, "\n")

## RMSE: 23057.4
cat("MAE :", ols_mae, "\n")

## MAE : 13013.17

```

```

# -----
# 6. Comparison table
# -----

comparison_table <- tibble(
  Model = c("OLS regression", "Bayesian hierarchical model"),
  Test_RMSE = c(ols_rmse, bayes_rmse),
  Test_MAE = c(ols_mae, bayes_mae)
)

print(comparison_table)

## # A tibble: 2 x 3
##   Model                      Test_RMSE Test_MAE
##   <chr>                      <dbl>    <dbl>
## 1 OLS regression            23057.   13013.
## 2 Bayesian hierarchical model 24099.   13441.

write_csv(comparison_table, "model_prediction_comparison.csv")

# -----
# 7. Save prediction results
# -----

prediction_results <- test_data %>%
  mutate(
    actual_price = actual_price,
    bayes_pred_price = bayes_pred_price,
    bayes_pred_lower = bayes_pred_lower,
    bayes_pred_upper = bayes_pred_upper,
    ols_pred_price = ols_pred_price
  )

write_csv(prediction_results, "used_car_prediction_results.csv")

# -----
# 8. Basic prediction plot
# -----

pdf("actual_vs_predicted_bayes.pdf", width = 7, height = 6)

plot(
  actual_price, bayes_pred_price,
  xlab = "Actual Price",
  ylab = "Bayesian Predicted Price",
  main = "Actual vs Predicted Used Car Prices: Bayesian Hierarchical Model",
  pch = 16,
  col = rgb(0, 0, 0, 0.35)
)

abline(0, 1, lwd = 2)

dev.off()

## pdf

```

```

## 2
pdf("actual_vs_predicted_ols.pdf", width = 7, height = 6)

plot(
  actual_price, ols_pred_price,
  xlab = "Actual Price",
  ylab = "OLS Predicted Price",
  main = "Actual vs Predicted Used Car Prices: OLS Regression",
  pch = 16,
  col = rgb(0, 0, 0, 0.35)
)

abline(0, 1, lwd = 2)

dev.off()

## pdf
## 2
cat("\nSaved files:\n")

##
## Saved files:
cat("- model_prediction_comparison.csv\n")

## - model_prediction_comparison.csv
cat("- used_car_prediction_results.csv\n")

## - used_car_prediction_results.csv
cat("- actual_vs_predicted_bayes.pdf\n")

## - actual_vs_predicted_bayes.pdf
cat("- actual_vs_predicted_ols.pdf\n")

## - actual_vs_predicted_ols.pdf

```